

Next.js Mini Guide

The App Router in Next.js 16: routing, Server vs Client Components, and three ways to handle data

Your to-do app already uses Next.js. This guide explains the framework itself and shows what else it can do, using small additions to the same app. Every example was built and tested in a copy of the project first.

- Part 1** What Next.js adds to React; the three commands
- Part 2** Routing: folders become URLs; special files
- Part 3** Server Components vs Client Components (the key idea)
- Part 4** Three ways to read and change data
- Part 5** Dynamic pages, loading and error screens
- Part 6** Route handlers, environment variables, and more
- Part 7** Common errors, exercises, interview questions

WATCH OUT

Next.js changes fast. This guide matches **Next.js 16** (the version in your project). Older tutorials may show things that no longer apply: `pages/` instead of `app/`, `getServerSideProps`, `middleware.ts` (now called `proxy.ts`), or `params.id` without `await`. The docs for your exact version ship inside the project at `node_modules/next/dist/docs/`.

Part 1: What Next.js adds to React

Job	React alone (e.g. Vite)	Next.js
Pages and URLs	Add a router library	Folders in <code>app/</code> become URLs
Backend / API	Separate server (Express)	<code>route.ts</code> files in the same project
Where code runs	Browser only	Server and browser, per component
First page load	Empty HTML, then JavaScript draws the page	HTML arrives already filled in (faster, better for search engines)
Reading a database	Only through an API	Server Components can read it directly

1 The three commands

Command	What it does	When
<code>npm run dev</code>	Development server with instant reload and detailed errors	While coding
<code>npm run build</code>	Checks types and compiles an optimised version into <code>.next/</code>	Before deploying; to catch errors
<code>npm run start</code>	Runs the built version (needs <code>build</code> first)	Production

`npm run build` ends with a list of every route. From a test copy of the app with this guide's examples added:

```
npm run build
```

```
Route (app)
├ ○ /
├ ○ /_not-found
├ f /api/search
├ f /api/todos
├ f /api/todos/[id]
├ f /todos
├ f /todos/[id]

○ (Static) prerendered as static content
f (Dynamic) server-rendered on demand
```

IN PLAIN ENGLISH

○ **Static** = built **once**, at build time, then the same HTML is sent to everyone (fast). **f Dynamic** = built **fresh on every request** (needed when data changes). Your home page is ○: its HTML is a shell, and the list is fetched later in the browser. Part 4 shows why this matters.

Part 2: Routing

The folder structure IS the URL structure.

2 Folders become URLs

File	URL
app/page.tsx	/
app/about/page.tsx	/about
app/todos/page.tsx	/todos
app/todos/[id]/page.tsx	/todos/1, /todos/2, ... (dynamic segment)
app/api/todos/route.ts	/api/todos (returns data, not a page)
app/(marketing)/pricing/page.tsx	/pricing : a folder in (brackets) groups files without adding to the URL

A folder is only a public URL if it contains a `page.tsx` or `route.ts`. Other files in it (components, helpers) are private.

3 Special file names

File	Purpose
page.tsx	The page for that URL
layout.tsx	Wraps the page <i>and all pages in sub-folders</i> ; stays on screen while navigating between them (nav bars, sidebars)

<code>route.ts</code>	API endpoint: export <code>GET</code> , <code>POST</code> , <code>PATCH</code> , <code>DELETE</code> ... (a folder can't have both <code>page.tsx</code> and <code>route.ts</code>)
<code>loading.tsx</code>	Shown instantly while the page in that folder loads
<code>error.tsx</code>	Shown if the page in that folder crashes
<code>not-found.tsx</code>	Custom 404 page (when <code>notFound()</code> is called or no route matches)
<code>proxy.ts</code> (project root)	Runs before matching requests, e.g. redirect logged-out users. Called <code>middleware.ts</code> before Next.js 16.

4 Moving between pages

TypeScript

```
import Link from "next/link";

<Link href="/todos">All todos</Link>           // like <a>, but no full page reload
<Link href={` /todos/${todo.id}`}>{todo.text}</Link>
```

Need	Use	Import from
A clickable link	<code><Link href="..."></code>	<code>next/link</code>
Navigate in code (after a click, in a Client Component)	<code>const router = useRouter();</code> <code>router.push("/todos")</code>	<code>next/navigation</code>
Send the user elsewhere from server code	<code>redirect("/login")</code>	<code>next/navigation</code>
Show the 404 page	<code>notFound()</code>	<code>next/navigation</code>

WATCH OUT

Import `useRouter` from `next/navigation` , **not** `next/router` . The second one is for the old `pages/` system, and many older tutorials still use it.

Part 3: Server vs Client Components

The single most important Next.js idea.

IN PLAIN ENGLISH

In the App Router, every component is a **Server Component** unless its file starts with `"use client"` . Server Components run on the server: they can read the database and secrets, then send finished HTML. **Client Components** also run in the browser, so they can react to clicks and typing.

Can it...	Server Component (default)	Client Component ("use client")
Read the database / secrets directly	✓	✗
Be an <code>async</code> function and <code>await</code> data	✓	✗
<code>useState</code> , <code>useEffect</code> , Zustand stores	✗	✓

onClick , onChange handlers	✗	✓
Browser things: window , localStorage	✗	✓
Add JavaScript to the download	No	Yes

a typical mix

```

app/todos/page.tsx      SERVER: reads the database, builds the list
└─ <TodoForm />         CLIENT ("use client"): the input box with useState
  <TodoList todos={...} /> SERVER: just displays what it's given
    └─ <DeleteButton />  CLIENT: needs onClick

```

- Put "use client" as low as possible: on the small interactive pieces, not the whole page.
- Everything a Client Component **imports** also becomes client code. That's why database code must never be imported there (see the error in Part 7).
- A Server Component can render a Client Component and pass it **props**, but only data (strings, numbers, plain objects, arrays), not functions. The exception is Server Actions (Part 4).

TIP

Your app's `page.tsx` is a Client Component (it needs `useState` , `Zustand` and clicks), so it gets its data via `fetch` to the API. Part 4 shows the alternative where the server reads the database itself.

Part 4: Three ways to read and change data

	A. Client + API route	B. Server Component	C. Server Action
Used for	Reading and changing	Reading	Changing (forms, buttons)
How	<code>fetch("/api/todos")</code> from the browser	<code>await findAllTodos()</code> in the component	<code><form action={addTodoAction}></code>
Needs <code>route.ts</code> ?	Yes	No	No
Other apps (mobile) can call it?	Yes : it's a real URL	No	No
In your app	✓ what you built	example below	example below

None is "the right one". A is the classic pattern every web developer must know (and it's what Express apps do). B and C are newer, need less code, and are what Next.js recommends inside one app. Knowing all three is a strong interview answer.

5 B. Read the database in a Server Component

A new page at `/todos` . No "use client" , no API route, no `fetch` , no `useEffect` :

app/todos/page.tsx

```
import Link from "next/link";
import { connection } from "next/server";
import { findAllTodos } from "@models/todo";
import { addTodoAction } from "./actions";

// A Server Component: no "use client", so it runs on the server
// and can read the database directly. No API route, no fetch.
export default async function TodosPage() {
  await connection(); // render on every request, not once at build time
  const todos = await findAllTodos();

  return (
    <main>
      <h1>Todos (Server Component)</h1>

      <form action={addTodoAction}>
        <input name="text" placeholder="New task..." />
        <button type="submit">Add</button>
      </form>

      <ul>
        {todos.map((t) => (
          <li key={t.id}>
            <Link href={` /todos/${t.id}`}>{t.text}</Link> {t.done ? "✓" : ""}
          </li>
        ))}
      </ul>
    </main>
  );
}
```

WATCH OUT

Why `await connection()` ? We tested it: without that line, the build marks `/todos` as ○ **Static**. Next.js would query the database **once while building** and show that same old list forever, even after you add tasks. `connection()` says "this page depends on the live request", which makes it **f Dynamic**.

npm run build, both versions

without connection():	○ /todos	← frozen at build time
with connection():	f /todos	← fresh on every visit

6 C. Change data with a Server Action

app/todos/actions.ts

```
"use server";

import { revalidatePath } from "next/cache";
import { createTodo } from "@models/todo";

// A Server Action: runs on the server, called straight from a <form>.
export async function addTodoAction(formData: FormData) {
  const text = formData.get("text");
  if (typeof text !== "string" || !text.trim()) return;
  await createTodo(text.trim());
  revalidatePath("/todos"); // show the fresh list
}
```

IN PLAIN ENGLISH

"use server" at the top makes every exported function a **Server Action**. Pass one to a form's `action`, and on submit Next.js sends the form data to the server, runs the function, and (thanks to `revalidatePath`) re-renders `/todos` with the new list. The input's `name="text"` is how `formData.get("text")` finds it. We tested it in the browser: typing a task and pressing Enter saved it and refreshed the list.

WATCH OUT

A Server Action is still a public endpoint underneath (a POST request anyone can send). **Validate the input** (like the `typeof` check above), and in a real app with logins, check the user is allowed inside every action.

IF YOU KNOW MONGOOSE / EXPRESS

Pattern A is exactly Express: routes + JSON. Patterns B and C have no Express equivalent; they remove the API layer when only your own pages need the data.

Part 5: Dynamic pages, loading and errors

7 A page per todo: app/todos/[id]/page.tsx

First add a model function (in `models/todo.ts`):

models/todo.ts (add)

```
// Find one todo by id (undefined if it doesn't exist)
export async function findTodoById(id: number): Promise<Todo | undefined> {
  const { rows } = await pool.query<Todo>("SELECT * FROM todos WHERE id = $1", [id]);
  return rows[0];
}
```

app/todos/[id]/page.tsx

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { findTodoById } from "@/models/todo";

type Props = { params: Promise<{ id: string }> };

// /todos/1, /todos/2, ... one page file for every id
export default async function TodoPage({ params }: Props) {
  const { id } = await params;
  const todo = await findTodoById(Number(id));
  if (!todo) notFound(); // shows the 404 page

  return (
    <main>
      <h1>{todo.text}</h1>
      <p>Status: {todo.done ? "done" : "not done"}</p>
      <Link href="/todos">← Back</Link>
    </main>
  );
}
```

Same idea as your API's `[id]` folder: `params` is a Promise, so `await` it, and the id arrives as text, so convert it with `Number()`. After `if (!todo) notFound();` TypeScript knows `todo` exists, because `notFound()` never returns.

8 loading.tsx and error.tsx

app/todos/loading.tsx

```
// Shown instantly while a page in this folder
is still loading
export default function Loading() {
  return <p>Loading todos...</p>;
}
```

app/todos/error.tsx

```
"use client"; // error pages must be Client
Components

// Shown instead of the page if it throws (e.g.
the database is down)
export default function Error({ error, retry }:
{ error: Error; retry: () => void }) {
  return (
    <main>
      <h2>Something went wrong</h2>
      <p>{error.message}</p>
      <button onClick={() => retry()}>Try
again</button>
    </main>
  );
}
```

Both apply to the folder they're in and its sub-folders (`/todos` and `/todos/[id]`). In Next.js 16 the error page's button prop is called `retry`; older tutorials call it `reset`.

TIP

A detail we found while testing: with a `loading.tsx`, a missing todo (`/todos/999999`) shows the "could not be found" page, but the HTTP status is **200**, not 404. The loading screen had already started sending the page, and a status can't change mid-way. Next.js adds a `noindex` tag so search engines ignore it. This is called a **soft 404**; it's fine for learning, but good to know about.

Part 6: Route handlers, env variables and more

9 Route handlers: query strings

Your API already uses route handlers. To read `?q=milk` from the URL, type the request as `NextRequest` and use `nextUrl.searchParams`:

`app/api/search/route.ts`

```
import { NextRequest, NextResponse } from "next/server";
import { searchTodos } from "@models/todo";

// GET /api/search?q=milk
export async function GET(req: NextRequest) {
  const q = req.nextUrl.searchParams.get("q") ?? "";
  const todos = await searchTodos(q);
  return NextResponse.json(todos);
}
```

`models/todo.ts (add)`

```
// Case-insensitive search. The % wildcards are added to the VALUE, not the SQL.
export async function searchTodos(q: string): Promise<Todo[]> {
  const { rows } = await pool.query<Todo>(`
    SELECT * FROM todos WHERE text ILIKE $1 ORDER BY id`,
    [`%${q}%`]);
  return rows;
}
```

tested

```
GET /api/search?q=MILK
[{"id":8,"text":"Buy milk","done":false}]
```

10 Environment variables

Rule	Example
Put them in <code>.env.local</code> (never committed to Git)	<code>DATABASE_URL=postgres://...</code>
Server code reads them with <code>process.env</code>	<code>process.env.DATABASE_URL</code>
The browser can't see them, unless the name starts with <code>NEXT_PUBLIC_</code>	<code>NEXT_PUBLIC_APP_NAME=Todo</code>
<code>NEXT_PUBLIC_</code> values are copied into the browser JavaScript at build time	So never put a password in a <code>NEXT_PUBLIC_</code> variable

After changing them, restart `npm run dev`

Our database pool is created once, with the old value

11 Sharing state between Client Components (Zustand)

components/ToDoCounter.tsx

```
"use client";

import { useTodoStore } from "@store/todoStore";

type Props = { label?: string };

// A second component reading the SAME Zustand store as the page.
// No props passed down: both just call useTodoStore().
export function ToDoCounter({ label = "Open" }: Props) {
  const openCount = useTodoStore((state) => state.todos.filter((t) => !t.done).length);
  return <p>{label}: {openCount}</p>;
}
```

Put `<ToDoCounter />` anywhere on the page and it updates whenever the list changes (tested: it showed "Open: 3" once the list loaded). The `(state) => ...` part is a **selector**: the component re-draws only when that number changes.

12 Smaller things worth knowing

Feature	How
Page title	<code>export const metadata = { title: "Todos" };</code> in a <code>page.tsx</code> or <code>layout.tsx</code> (Server Components only)
Global CSS	<code>import "../globals.css";</code> in <code>app/layout.tsx</code>
Per-component CSS	<code>Button.module.css</code> + import styles from <code>"./Button.module.css"</code> , then <code>className={styles.primary}</code>
Images	<code><Image src="/logo.png" width={100} height={40} alt="" /></code> from <code>next/image</code> ; files go in <code>public/</code>
Static files	Anything in <code>public/</code> is served at <code>/</code> : <code>public/logo.png</code> becomes <code>/logo.png</code>

Part 7: Errors, exercises, interviews

Common errors

You see	Cause and fix
Module not found: Can't resolve 'dns' (or 'fs', 'net', 'tls'), with "Client Component Browser" in the trace	Database code got imported into a <code>"use client"</code> file (we caused this on purpose to capture the message). The browser has no network sockets or files. Call an API route or a Server Action instead, and import types with <code>import type</code> .

An error saying <code>useState</code> / <code>useEffect</code> / <code>onClick</code> only works in a Client Component	Add <code>"use client";</code> at the top of that file, or move the interactive part into its own small client component.
An error saying <code>async</code> Client Components aren't supported	Only Server Components can be <code>async</code> . In a Client Component, load data in <code>useEffect</code> or a store action.
The page shows old data after adding items	The page was built as static (O). Add <code>await connection()</code> , or call <code>revalidatePath</code> after changes.
A "hydration" error: the server's HTML didn't match the browser's	The server HTML and the browser's first draw differ, e.g. rendering <code>Date.now()</code> , <code>Math.random()</code> or <code>localStorage</code> values directly. Read browser-only values inside <code>useEffect</code> .
Type error on <code>params.id</code>	<code>params</code> is a Promise in current Next.js: <code>const { id } = await params;</code>
<code>process.env.X</code> is undefined in the browser	Only <code>NEXT_PUBLIC_</code> variables reach the browser. Also restart the dev server after editing <code>.env.local</code> .

Exercises (build them in your app)

- About page.** Create `app/about/page.tsx` with a `<Link>` back to `/`, and a link to it from the home page.
- Server-rendered list.** Add `app/todos/page.tsx` from Part 4. Then remove `await connection()`, run `npm run build`, and find `o /todos` in the output. Put it back.
- Detail page.** Add `findTodoById` and `app/todos/[id]/page.tsx`. Visit `/todos/1` and `/todos/999999`.
- See the loading screen.** Add `loading.tsx`, then temporarily put `await new Promise((r) => setTimeout(r, 2000));` at the start of the `/todos` page.
- See the error screen.** Add `error.tsx`, then temporarily `throw new Error("Database is down")` in the page. Click Try again.
- Mark done with a Server Action.** In `actions.ts`, write `toggleTodoAction(formData)` that reads a hidden `<input type="hidden" name="id" value={t.id} />`, calls `updateTodoDone`, and revalidates `/todos`. Add a small form with a button to each list item.
- Search page.** Add the search API from Part 6, then a client page `app/search/page.tsx` with an input that fetches `/api/search?q=...` as you type.

Interview questions, short answers

Question	Short answer
What is Next.js, versus React?	React builds UI components. Next.js is a framework on top: file-based routing, server rendering, API routes, and build optimisation in one project.
Server vs Client Components?	Server Components (the default) run on the server, can read data directly, and send no JavaScript for themselves. Client Components (<code>"use client"</code>) also run in the browser and handle interactivity and state.
Static vs dynamic rendering?	Static pages are built once at build time and reused (fast). Dynamic pages are rendered per request (fresh data). The build output marks them <code>O</code> and <code>f</code> .

How do you fetch data in the App Router?	In Server Components, by awaiting it directly (database or <code>fetch</code>). In Client Components, via route handlers (API) with <code>fetch</code> , often in a store or hook.
What are Server Actions?	" <code>use server</code> " functions that client forms and buttons can call directly. Used for changing data; they run on the server and can revalidate pages.
Route handler vs Server Action?	A route handler is a real HTTP API, usable by anyone (mobile apps, other services). A Server Action is for your own app's forms and buttons, with less code.
What is hydration?	The browser attaching React's interactivity to HTML the server already sent. A mismatch between the two causes hydration errors.
How are secrets kept safe?	They live in <code>.env.local</code> and are only readable by server code. Anything prefixed <code>NEXT_PUBLIC_</code> is public.

INTERVIEW ANGLE

Explaining your own project is the best answer to "walk me through something you built": *"The page is a Client Component using a Zustand store. The store calls REST route handlers, which validate input and call a model layer that runs parameterized SQL through a pg pool. I also know the Server Component and Server Action approach, which removes the API layer when only my own pages need the data."*